



Universität
Bremen

Fachbereich 3: Mathematik und Informatik

Entwicklung eines Archetype-ECS und einer WebGPU-Engine zur Echtzeit-Simulation von Millionen Entities einer planetaren Zivilisation im Browser

Artisan: Ein Archetype-ECS mit WebAssembly und WebGPU

Bericht zur Independent Study

im Studiengang Digitale Medien

Eingereicht von:

Aaron Postels
Matrikelnr.: 6277067
postels@uni-bremen.de

Betreuer / Gutachter:

Prof. Dr.-Ing. Udo Frese

Bremen, den 29. August 2026

Kurzfassung

Diese Arbeit untersucht, wie sich rechen- und grafikintensive Spiele und Echtzeitsimulationen effizient im Browser umsetzen lassen. Dafür werden ein datenorientiertes Entity-Component-System (ECS), Rust, WebAssembly und WebGPU zu einer gemeinsamen Architektur verbunden. Eine planetare Zivilisationssimulation dient als Anwendungsbeispiel; gezielte Benchmarks bewerten die zentralen Datenpfade.

Das Web bietet heute leistungsfähige Schnittstellen für komplexe Anwendungen. WebAssembly führt Rust-Code effizient im Browser aus. WebGPU ermöglicht einen modernen, direkten Zugriff auf die Grafikkarte. Anwendungen im Web starten ohne Installation über eine URL, lassen sich zentral aktualisieren und auf unterschiedlichen Systemen nutzen. Game Engines behandeln den Browser jedoch meist nur als zusätzliches Exportziel. Dadurch bleiben die Möglichkeiten moderner Webtechnologien oft ungenutzt.

Artisan ist die dafür entwickelte browsernative Engine-Architektur. Der Prototyp verbindet die derzeit leistungsfähigsten verfügbaren Techniken und untersucht, wie Spiele und Simulationen im Browser künftig aufgebaut sein könnten. Ziel ist eine Leistung nahe an nativen, installierten Anwendungen statt eines nachträglichen Browserexports mit technischen Altlasten.

Interaktive Online-Version. Diese Arbeit ist zusätzlich als Web-Dokumentation mit ausführbaren Demos verfügbar: ducklin.de/projects/artisan-paper

Inhaltsverzeichnis

1	Einleitung und Projektthema	1
1.1	Warum Spiele im Browser?	1
1.2	Die zentrale Frage	1
1.3	Warum ein datengetriebener Ansatz?	1
1.4	Artisan: der Kern des Projekts	2
1.5	Der konkrete Prüfstein: Vivarium	2
1.6	Wie der Ansatz geprüft wird	3
1.7	Ergebnis in Kürze	3
2	Grundlagen	4
2.1	Wie ein ECS die Spielwelt ordnet	4
2.2	Wo die Daten tatsächlich liegen	4
2.2.1	Sparse Set	5
2.2.2	Archetype-Modell	5
2.3	Warum zusammenhängende Daten schneller sind	5
2.4	Wie Rust in den Browser kommt	6
2.5	Parallele Ausführung im Browser	7
2.6	Wie WebGPU das Bild auf den Bildschirm bringt	7
2.7	Viele Objekte mit wenigen Befehlen zeichnen	8
3	Architektur und Umsetzung	9
3.1	Drei Schichten	9
3.2	Bausteine der Game Engine	9
3.3	Wo die Komponenten im Speicher liegen	9
3.4	Wie Daten abgefragt werden	10
3.5	Wer wann rechnen darf	10
3.6	Wie Daten zu JavaScript kommen	10
3.7	Wie WebGPU zeichnet	10
4	Anwendungsfall: Vivarium	11
4.1	Vom Mesh zur Welt	11
4.2	Höhe, Klima und Ressourcen	11
4.3	Der Simulationsgraph	11
4.4	Autonome Siedler	12
4.5	Was Vivarium zeigt	12
5	Evaluation und Performanzmessungen	13
5.1	Methodik	13
5.1.1	Auswahl der Tests	13
5.1.2	Durchführung	13
5.1.3	Auswertung	13
5.2	ECS-Messreihe	14
5.2.1	Gesamtergebnis	14
5.3	Messdiagramme	14
5.3.1	Iteration nach Entity-Zahl (Sweep A)	14
5.3.2	Archetype-Fragmentierung (Sweep B)	15

5.4 Alle 34 Messkategorien im Detail	16
5.5 Wo Artisan verliert	17
5.6 Testumgebung	17
6 Demos und Demonstratoren	18
6.1 Demoübersicht	18
7 Grenzen	20
7.1 API- und Speicherverträge	20
7.2 Laufzeit und Korrektheit	20
7.3 Aussagekraft der Benchmarks	21
7.4 Grenzen als Web-Engine	21
8 Fazit	22
8.1 Antwort auf die Ausgangsfrage	22
8.2 Was der Prototyp belegt	22
8.3 Bewertung der Architektur	23
8.4 Ausblick	23
Literaturverzeichnis	24
A Anhang: Nutzung KI-basierter Anwendungen	25
A.1 Verwendete Werkzeuge	25
A.2 Art der Unterstützung	25
A.3 Eigenanteil und Verantwortung	25

Abkürzungsverzeichnis

AoS	Array of Structures (klassische objektorientierte Speicheranordnung)
API	Application Programming Interface (Programmierschnittstelle)
COEP	Cross-Origin-Embedder-Policy (Sicherheitsheader für geteilten Speicher)
COOP	Cross-Origin-Opener-Policy (Sicherheitsheader für geteilten Speicher)
CPU	Central Processing Unit (Hauptprozessor)
ECS	Entity-Component-System (datenorientiertes Architekturmuster)
FPS	Frames per Second (Bilder pro Sekunde)
GPU	Graphics Processing Unit (Grafikprozessor)
HTTP	Hypertext Transfer Protocol (Übertragungsprotokoll des Webs)
ID	Identifizier (eindeutige Kennnummer)
KI	Künstliche Intelligenz
MSVC	Microsoft Visual C++ Compiler
OOP	Object-Oriented Programming (objektorientierte Programmierung)
SoA	Structure of Arrays (spaltenorientierte Speicheranordnung)
URL	Uniform Resource Locator (Webadresse)
VRAM	Video Random Access Memory (Grafikkartenspeicher)
Wasm	WebAssembly (kompaktes Binärformat für hardwarenahe Web-Ausführung)
W3C	World Wide Web Consortium (Standardisierungsgremium des Webs)
WebGL	Web Graphics Library (ältere Browser-Grafikschnittstelle)
WebGPU	moderne Browser-Schnittstelle für Grafik und GPU-Berechnungen
WGSL	WebGPU Shading Language (Programmiersprache für WebGPU-Shader)

1 Einleitung und Projektthema

Diese Arbeit untersucht, wie sich rechen- und grafikintensive Spiele und Echtzeitsimulationen effizient im Browser umsetzen lassen. Dafür werden ein datenorientiertes Entity-Component-System (ECS), Rust, WebAssembly und WebGPU zu einer gemeinsamen Architektur verbunden. Eine planetare Zivilisationssimulation dient als Anwendungsbeispiel; gezielte Benchmarks bewerten die zentralen Datenpfade.

1.1 Warum Spiele im Browser?

Das Web bietet heute leistungsfähige Schnittstellen für komplexe Anwendungen. WebAssembly führt Rust-Code effizient im Browser aus. WebGPU ermöglicht einen modernen, direkten Zugriff auf die Grafikkarte.

Anwendungen im Web starten ohne Installation über eine URL, lassen sich zentral aktualisieren und auf unterschiedlichen Systemen nutzen. Game Engines behandeln den Browser jedoch meist nur als zusätzliches Exportziel. Dadurch bleiben die Möglichkeiten moderner Webtechnologien oft ungenutzt.

Artisan ist die dafür entwickelte browsernative Engine-Architektur. Der Prototyp verbindet die derzeit leistungsfähigsten verfügbaren Techniken und untersucht, wie Spiele und Simulationen im Browser künftig aufgebaut sein könnten. Ziel ist eine Leistung nahe an nativen, installierten Anwendungen statt eines nachträglichen Browserexports mit technischen Altlasten.

1.2 Die zentrale Frage

Wie weit lässt sich die Leistung datenintensiver Spiele und Echtzeitsimulationen im Browser treiben, wenn sie von Beginn an auf moderne Webtechnologien und eine datenorientierte Architektur ausgelegt werden? Untersucht wird, wie schnell der Browser große Mengen gleichartig strukturierter Daten verarbeiten, zwischen WebAssembly und JavaScript bereitstellen und über WebGPU darstellen kann.

Die planetare Zivilisationssimulation bildet dafür einen zusammenhängenden Anwendungsfall. Sie verbindet prozedurale Geometrie, lokale Ressourcen, individuelle Siedler und Rendering in einer laufenden Anwendung. Die Grenzen der einzelnen Datenpfade werden davon getrennt in gezielten ECS-, Brücken- und Renderingtests untersucht.

1.3 Warum ein datengetriebener Ansatz?

Viele klassische Engines modellieren eine Spielwelt durch objektorientierte Strukturen. Das ist vertraut, verteilt die Daten einer großen Simulation jedoch häufig über den Speicher. Für dieses Projekt wurde deshalb ein Entity-Component-System gewählt, das Identität, Daten und Verhalten voneinander trennt. Gleichartige Daten liegen zusammenhängend im Speicher und lassen sich in dichten Schleifen verarbeiten.

Dieser Aufbau passt nicht nur zu großen Simulationen. Er schafft auch eine klare, maschinenlesbare Beschreibung der Welt. Systeme arbeiten auf fest definierten Datenmengen, Abhängigkeiten werden sichtbar und Arbeit kann gezielter parallelisiert werden. Mit der zunehmenden Nutzung ge-

nerativer KI erscheint eine solche Trennung von Daten und Verhalten zusätzlich sinnvoll: Werkzeuge können strukturierte Bausteine leichter zusammensetzen und prüfen als tiefe, projektspezifische Klassenhierarchien.

1.4 Artisan: der Kern des Projekts

Artisan ist eine von Grund auf für den Browser entwickelte Game Engine. Im Fokus stehen ihre Architektur, die Datenverarbeitung und der Weg bis zur Darstellung. Die Engine stellt Entities und Komponenten, Queries und Systeme, Zeitsteuerung, Eingaben, Transformationen, Assets sowie 2D- und 3D-Rendering bereit.

Spielspezifische Komponenten und Systeme werden in Rust ergänzt und gemeinsam mit dem Engine-Kern nach WebAssembly übersetzt. JavaScript bindet die Anwendung in den Browser ein und steuert den WebGPU-Renderer. Die planetare Zivilisationssimulation **Vivarium** ergänzt darauf aufbauend eigene Komponenten und Systeme für Siedler, Bedürfnisse und Stämme. Die weiteren Demos verwenden dieselbe Engine für andere Arbeitslasten. Wie schnell die dabei untersuchten Datenpfade arbeiten, wird in der Evaluation getrennt betrachtet.

Der Datenweg. Rust und WebAssembly aktualisieren zusammenhängende Komponentenspalten. JavaScript liest ausgewählte Bereiche direkt aus diesem Speicher. Der WebGPU-Renderer überträgt die sichtbaren Instanzen anschließend in GPU-Buffer und zeichnet sie in möglichst wenigen Batches.

1.5 Der konkrete Prüfstein: Vivarium

Vivarium ist das Anwendungsbeispiel für die eingesetzten Technologien. Die Demo setzt die im Titel genannte planetare Zivilisationssimulation um und führt die zentralen Teile von Artisan in einer vollständigen Anwendung zusammen.

Im Mittelpunkt steht, wie Artisan die dafür benötigte Simulation, Datenverwaltung und Darstellung im Browser ermöglicht. Dazu gehören das Speichermodell des ECS, die Verbindung zwischen WebAssembly und JavaScript und der Weg von den Simulationsdaten zum Zeichenbefehl. Vivarium verbindet diese Bereiche in einer zusammenhängenden Spielanwendung.

Auch die Zahl im Titel ist einzuordnen. Eine Million Entities werden in den Messungen erreicht, dort allerdings mit wenigen Komponenten je Entity und ohne vollständige Simulationslogik. Die Zivilisationssimulation läuft mit deutlich weniger Siedlern, dafür mit erheblich mehr Logik pro Entity. Die Untersuchung zeigt beide Zahlen getrennt, statt sie zu einer zusammenzufassen.

1.6 Wie der Ansatz geprüft wird

Eine Engine allein zu messen sagt wenig. Ohne Vergleichswert ist unklar, ob eine Millisekunde für eine Aufgabe viel oder wenig ist. Deshalb laufen dieselben Aufgaben auf zwei etablierten Systemen mit:

1. **Referenz 1: Bevy 0.18.1:** Eine bekannte Open-Source-Game-Engine, die wie Artisan in Rust geschrieben ist. Sie läuft normalerweise nativ auf dem Rechner, also ohne Browser. Weil dieselbe Programmiersprache und ein verwandtes Speichermodell zum Einsatz kommen, lässt sich ein Unterschied nicht einfach auf die Sprache schieben. Bevy ist damit die aussagekräftigste Referenz.
2. **Referenz 2: Flecs:** Flecs ist ein in C geschriebenes Open-Source-ECS und gilt als eines der schnellsten ECS-Frameworks [1]. Es konzentriert sich auf die effiziente Verwaltung großer Mengen von Entities und ergänzt Bevy damit als spezialisierte zweite Referenz.

1.7 Ergebnis in Kürze

Über alle Messkategorien hinweg liegt Artisan vorn, aber nicht überall:

- **Artisan:** geometrisches Mittel **1,05x**, schnellste Engine in **20 von 34** Kategorien.
- **Flecs:** geometrisches Mittel **1,37x**, schnellste Engine in **12 von 34** Kategorien.
- **Bevy 0.18.1:** geometrisches Mittel **1,50x**, schnellste Engine in **2 von 34** Kategorien.

Der Wert fasst zusammen, wie weit eine Engine im Mittel vom jeweils schnellsten Ergebnis einer Kategorie entfernt liegt. 1,00 bedeutet, dass sie in allen Kategorien am schnellsten war; höhere Werte bedeuten einen größeren durchschnittlichen Abstand. Für diese Verhältnisse wird das geometrische Mittel verwendet: Es mittelt die Faktoren durch Multiplikation und anschließendes Wurzelziehen, damit nicht eine einzelne Kategorie mit besonders großem Zahlenwert die Zusammenfassung bestimmt.

2 Grundlagen

Artisan verbindet drei Ideen. Ein Entity-Component-System (ECS) ordnet die Spielwelt als Daten. Rust und WebAssembly führen rechenintensive Logik schnell im Browser aus. WebGPU zeichnet das Ergebnis mit moderner Grafikhardware. Dieses Kapitel erklärt, wie diese drei Bausteine zusammenarbeiten und warum sie gewählt wurden.

2.1 Wie ein ECS die Spielwelt ordnet

Viele Spieleentwickler lernen objektorientierte Programmierung (OOP). Dort besitzt eine Spielfigur Eigenschaften wie Position, Lebenspunkte und Darstellung und bringt Methoden mit, um sich zu bewegen oder Schaden zu nehmen. In einem Entity-Component-System wird dieser Aufbau in drei getrennte Rollen zerlegt [2]:

- **Entity:** Ein Entity ist eine reine Kennnummer, vergleichbar mit einer Zeilennummer in einer Tabelle. Es enthält weder eigene Daten noch Methoden.
- **Komponente (Component):** Eine Komponente speichert Daten zu einem bestimmten Aspekt. Eine Position speichert die Koordinaten, eine Health-Komponente die Lebenspunkte. Komponenten enthalten keine Programmlogik.
- **System:** Ein System enthält die Programmlogik. Es fragt nach allen Entities, die bestimmte Komponenten besitzen, und verarbeitet diese Daten nacheinander. Ein Bewegungssystem sucht beispielsweise nach allen Entities mit Position und Velocity und aktualisiert deren Position.

Entity	Position	Velocity	Health	Flying
E1 · Drache	x: 10, y: 20	vx: 1, vy: 0	100	ja
E2 · Fledermaus	x: 5, y: 15	vx: 2, vy: 1	30	ja
E3 · Goblin	x: 0, y: 0	vx: 0, vy: 0	45	nein
E4 · Pfeil	x: 12, y: 8	vx: 5, vy: 2	-	-
E5 · Wegpunkt	x: 20, y: 0	-	-	-

Ein Bewegungssystem fragt nach Position und Velocity und findet E1, E2, E3 und E4. Ein Flugsystem fragt zusätzlich nach Flying und findet nur E1 und E2. Der Wegpunkt E5 besitzt keine Geschwindigkeit und wird vom Bewegungssystem übersprungen.

Der Vorteil zeigt sich, wenn neue Spielmechaniken hinzukommen: Soll ein Pfeil brennen, erhält er eine Burning-Komponente. Weder Pfeil noch Bewegungssystem müssen dafür umgeschrieben werden. Das brennende Verhalten steckt in einem eigenen System, das nur nach Burning sucht.

2.2 Wo die Daten tatsächlich liegen

Die Tabelle oben beschreibt das logische Modell. Für die Ausführungsgeschwindigkeit ist jedoch entscheidend, wie die Komponentenwerte tatsächlich im Speicher liegen. Genau das entscheidet darüber, wie schnell ein System durch tausende Entities laufen kann. Zwei Speichermodelle haben sich durchgesetzt:

2.2.1 Sparse Set

Ein **Sparse Set** hält für jeden Komponententyp ein dicht gepacktes Array mit den tatsächlich vorhandenen Werten, dazu eine Indextabelle, die von einem Entity zur Position in diesem Array führt [3]. Einfügen, Entfernen und der direkte Zugriff über ein bekanntes Entity sind dabei sehr günstig. Der Nachteil zeigt sich, sobald eine Query mehrere Komponententypen gleichzeitig braucht: Für jedes Entity muss dann geprüft werden, ob es in allen beteiligten Sets vorkommt, bevor überhaupt gerechnet werden kann.

2.2.2 Archetype-Modell

Ein **Archetype-Modell** geht umgekehrt vor [1], [4]. Alle Entities mit exakt derselben Kombination von Komponenten landen zusammen in einer Tabelle, ihrem Archetype. Innerhalb dieser Tabelle liegt jeder Komponententyp in einer eigenen, zusammenhängenden Spalte. Eine Query nach Position und Velocity muss dadurch nur die Archetypes finden, die beide Spalten besitzen, und kann sie danach ohne weitere Prüfung Zeile für Zeile durchlaufen.

Der Preis dafür. Der Vorteil des Archetype-Modells ist die dichte, ununterbrochene Iteration. Beahlt wird er bei strukturellen Änderungen: Verliert E1, der Drache, seine Flugfähigkeit, ändert sich seine Signatur, also die genaue Kombination seiner Komponenten. Es muss komplett von Archetype A nach Archetype B umziehen. Dabei wandert nicht nur die entfernte Komponente Flying, sondern auch Position, Velocity und Health in die neue Tabelle mit.

Beide Modelle schließen sich nicht gegenseitig aus. Bevy speichert standardmäßig in Tabellen und bietet Sparse Sets als Alternative für Komponenten an, die häufig hinzugefügt und entfernt werden [4]. Flecs verwendet ebenfalls Archetypes, erlaubt aber zusätzlich Sparse-Komponenten außerhalb der Tabellen [1]. Artisan besitzt im aktuellen Stand nur das Archetype-Modell, keine solche Ausweichmöglichkeit.

64-Byte CPU-Cache-Line beim Durchlauf eines Bewegungssystems (Position und Velocity)

1. Klassische Objektspeicherung (Array of Structures · AoS)

Beispielrechnung: 37,5 % (24 von 64 Bytes genutzt)



2. Archetype-Spaltenspeicher (Structure of Arrays · SoA)

Nutzungsgrad: 100 % (Sequenzieller Datenstrom)

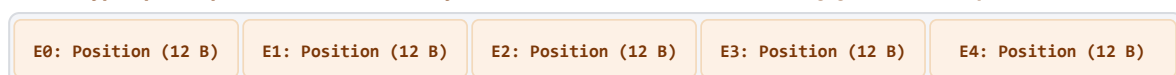


Abbildung 1: Speicheranordnung und Cache-Line-Auslastung: Beispielrechnung für ein 64-Byte-Layout (AoS mit 37,5 % vs. Archetype-Spalten mit 100 % Auslastung).

2.3 Warum zusammenhängende Daten schneller sind

Zwischen Prozessor und Arbeitsspeicher liegt ein kleiner, schneller Zwischenspeicher, der CPU-Cache. Der Prozessor lädt Werte daraus nicht einzeln, sondern in Blöcken benachbarter Bytes, den Cache Lines. Werden Werte nacheinander im Speicher gelesen, liegen die nächsten benötigten Werte oft schon im Cache, bevor sie ausdrücklich angefragt werden. Dieses Prinzip heißt räumliche Lokalität.

Archetype-Spalten sind für genau dieses Muster gebaut: Alle Werte eines Komponententyps liegen direkt hintereinander. Ein Bewegungssystem, das nur `Position` und `Velocity` braucht, lädt dadurch keine Namen, Materialien oder Kollisionsformen mit, nur weil diese zufällig zum selben Entity gehören (siehe Abbildung 1).

Das erklärt, warum ein Vorteil zu erwarten ist. Es belegt ihn aber noch nicht: Wie viel davon in der Praxis ankommt, hängt zusätzlich von der Query-Implementierung, der Zahl der Archetypes, der Breite der Komponenten, der Parallelisierung und der konkreten Hardware ab.

2.4 Wie Rust in den Browser kommt

Rust in Kürze. Rust ist eine kompilierte, statisch typisierte Systemprogrammiersprache ohne Garbage Collector [5]. Der Compiler verfolgt, wem ein Wert gehört und wie lange Verweise darauf gültig sind. Dadurch verbindet Rust eine ähnlich genaue Kontrolle über Speicher und Laufzeitkosten wie C oder C++ mit Prüfungen, die viele ungültige Speicherzugriffe und konkurrierende Schreibzugriffe bereits beim Kompilieren verhindern.

Warum Rust? Artisan hätte grundsätzlich auch in C oder C++ entwickelt werden können; beide Sprachen lassen sich nach WebAssembly übersetzen und erlauben ebenfalls ein kontrolliertes Speicherlayout. Rust wurde nicht gewählt, weil es automatisch schneller wäre, sondern wegen seiner zusätzlichen Sicherheitsprüfungen und seines passenden Werkzeug-Ökosystems. Cargo verwaltet Build und Abhängigkeiten, `wasm-bindgen` verbindet Rust mit JavaScript und `wasm-bindgen-rayon` überträgt Rayons Parallelisierungsmodell auf Web Workers. Derselbe Engine-Kern lässt sich als WebAssembly-Modul für den Browser und nativ für Tests und Benchmarks kompilieren.

JavaScript ist dynamisch typisiert: Eine Variable kann zur Laufzeit Werte unterschiedlicher Typen aufnehmen. Außerdem verwaltet die Sprache Speicher automatisch. Moderne JavaScript-Engines übersetzen häufig ausgeführten Code zwar zur Laufzeit in Maschinencode, also direkt ausführbare Prozessorbefehle. Diese Optimierung beruht jedoch auf Annahmen über die beobachteten Typen und kann bei Änderungen wieder verworfen werden. Auch den Zeitpunkt der Garbage Collection, der automatischen Freigabe nicht mehr benötigten Speichers, bestimmt nicht die Anwendung.

Für ein Archetype-ECS ist vor allem der fehlende Zugriff auf das Speicherlayout ein Problem: Die ganze Idee der Archetype-Spalten beruht darauf, Komponenten als zusammenhängenden Speicherbereich zu behandeln, den auch eine andere Sprache adressieren kann. Gesucht ist deshalb kein Ersatz für JavaScript als Ganzes, sondern ein Ausführungsformat für genau den rechen- und speicherintensiven Teil, den Engine-Kern.

WebAssembly in Kürze. WebAssembly (oft als Wasm abgekürzt) ist ein standardisiertes, kompaktes Binärformat, das von allen modernen Browsern ausgeführt werden kann [6], [7]. Rust-Code wird vorab in dieses Format übersetzt. Der Browser lädt das fertige Wasm-Modul, prüft es und übersetzt es einmalig in Maschinencode.

WebAssembly arbeitet auf einem zusammenhängenden Speicherbereich, dem linearen Speicher [8]. Dieser Speicher gehört dem Wasm-Modul; JavaScript kann ihn jedoch als sogenanntes TypedArray lesen und beschreiben, also als Array mit festem Datentyp wie `Float32Array`. Das ist die Brücke zwischen beiden Welten: Rust schreibt seine Komponentenspalten in den linearen Speicher, JavaScript liest die Werte direkt an derselben Stelle aus.

2.5 Parallele Ausführung im Browser

Standardmäßig läuft JavaScript in einem einzelnen Ausführungspfad, dem sogenannten Hauptthread. Wenn eine aufwendige Berechnung den Hauptthread blockiert, friert die Seite ein: Schaltflächen reagieren nicht mehr, Animationen stocken.

Für parallele Arbeit stellt der Browser **Web Worker** bereit. Jeder Worker ist ein eigener Hintergrundthread mit eigenem Speicher. Sollen mehrere Worker an denselben Daten arbeiten, teilen sie sich einen gemeinsamen Speicherbereich, einen `SharedArrayBuffer`. Zugriffe darauf werden über atomare Operationen synchronisiert, die verhindern, dass zwei Threads denselben Wert gleichzeitig unkontrolliert verändern [9].

In nativem Rust ist **Rayon** die Standardbibliothek für datenparallele Schleifen [10]. Statt Threads von Hand zu starten und zu synchronisieren, verteilt Rayon anfallende Arbeit automatisch auf verfügbare Prozessorkerne (**Work-Stealing**). Über die Bibliothek `wasm-bindgen-rayon` [11] nutzt Artisan genau dieses Modell im Browser: Der Engine-Kern verwendet dieselben parallelen Schleifen wie im nativen Programm, die Arbeit wird auf einen Pool von Web Workern verteilt.

Voraussetzung im Browser. Aus Sicherheitsgründen, namentlich zum Schutz vor Seitenkanalangriffen wie **Spectre**, erlauben Browser `SharedArrayBuffer` nur auf Seiten, die über spezielle HTTP-Header als isoliert gekennzeichnet sind (**Cross-Origin Isolation** via COOP und COEP). Fehlen diese Header, läuft die Ausführung auf einen einzelnen Thread zurück.

2.6 Wie WebGPU das Bild auf den Bildschirm bringt

WebGL in Kürze. WebGL brachte 2011 3D-Grafik in den Browser [12]. Es basiert auf OpenGL ES 2.0/3.0 und arbeitet als globale Zustandsmaschine: Vor jedem Zeichenbefehl müssen Shader, Texturen und Einstellungen einzeln nacheinander gesetzt werden. Jeder dieser Schritte ist ein separater Funktionsaufruf zwischen JavaScript und dem Grafiktreiber. Bei zehntausenden Objekten pro Bild wird diese Kommunikation zum Engpass.

WebGPU in Kürze. WebGPU ist der moderne Nachfolger von WebGL [13]. Es orientiert sich an modernen Grafik-Schnittstellen wie Vulkan, Metal und DirectX 12. Statt vieler einzelner Befehle zur Laufzeit werden Pipeline-Zustände, also Shader, Puffer-Layouts und Mischmodi, vorab in einem unveränderlichen Pipeline-Objekt zusammengefasst. Befehle werden gebündelt in einem sogenannten Command Buffer aufgezeichnet und in einem einzigen Schritt an die Grafikkarte übergeben.

WebGPU bringt außerdem eigene Rechenprogramme auf die Grafikkarte, sogenannte **Compute Shader**. Sie ermöglichen es, Berechnungen direkt auf der GPU auszuführen, ohne den Umweg über die CPU. Shaders werden in einer eigenen, typischeren Sprache geschrieben, der **WebGPU Shading Language (WGSL)** [14].

2.7 Viele Objekte mit wenigen Befehlen zeichnen

Um viele Objekte darzustellen, nutzt Artisan **GPU-Instancing**. Beim Instancing wird die Geometrie eines Modells (z. B. ein Würfel aus 36 Dreiecks-Indizes) einmalig im Speicher der Grafikkarte abgelegt. Soll das Modell 100.000-mal an verschiedenen Orten gezeichnet werden, übergibt die CPU nicht 100.000 einzelne Zeichenbefehle, sondern einen einzigen Befehl zusammen mit einem Array von Transformationsdaten. Die Grafikkarte liest für jede Instanz die passende Position und Ausrichtung aus diesem Puffer und zeichnet alle Kopien in einem Durchlauf.

Die Auswahllogik entscheidet, welche Entities sichtbar sind und welche Entities gemeinsam instanziiert gezeichnet werden können. In Artisan läuft diese Auswahl im Rust- und Wasm-Kern, JavaScript setzt anschließend nur die daraus abgeleiteten Draw Calls ab. Die Pipeline ist damit CPU-gesteuert und nutzt GPU-Instancing. Bei einer vollständig GPU-gesteuerten (GPU-driven) Pipeline bleiben dagegen auch Simulation, Sichtbarkeitsprüfung und Instanzdaten auf der Grafikkarte.

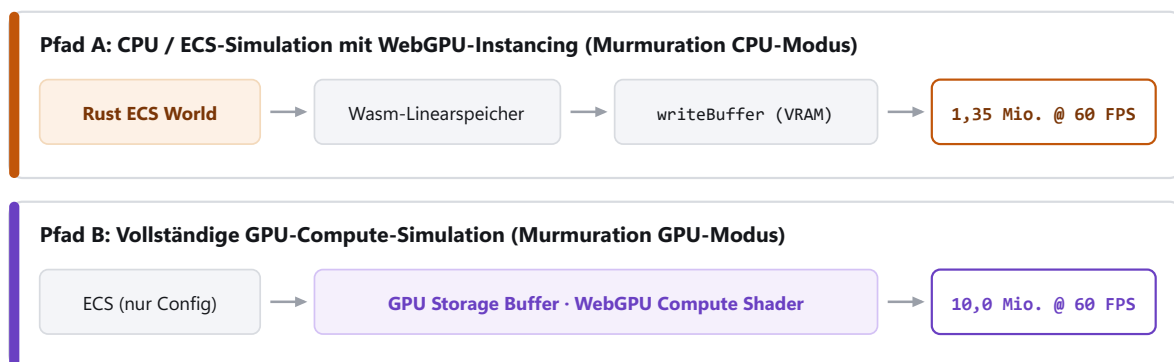


Abbildung 2: Simulationspfade im Vergleich: CPU-basierte ECS-Logik mit WebGPU-Instancing vs. direkte GPU-Compute-Simulation.

Die Demo **Murmuration** (siehe Abbildung 2) zeigt CPU- und GPU-Simulation im Vergleich:

- Bei der CPU-Simulation ist jeder Würfel ein ECS-Entity und wird in Rust aktualisiert. Danach zeichnet WebGPU alle Würfel per Instancing.
- Bei der GPU-Simulation liegen die Daten der Würfel direkt in einem GPU-Buffer und werden dort von einem Compute Shader aktualisiert. Das ECS enthält dann nur die Konfiguration der gesamten Simulation.

Auf dem Testsystem mit Intel Core i5-13600KF und RTX 4070 erreicht die Demo bei 60 FPS etwa 1,35 Millionen gleichzeitig simulierte und dargestellte ECS-Entities. Im GPU-Compute-Pfad sind ohne Frustum Culling etwa 10 Millionen Würfel möglich. Frustum Culling überspringt Würfel außerhalb des sichtbaren Kamerabereichs; mit dieser Sichtbarkeitsprüfung waren etwa 1,2- bis 4-mal so viele Würfel möglich.

3 Architektur und Umsetzung

Dieses Kapitel beschreibt, wie Artisan aufgebaut ist. Der Schwerpunkt liegt auf der Datenverarbeitung im Rust-Kern, der Verbindung zu JavaScript und dem Weg von den Simulationsdaten zum Bild auf dem Bildschirm.

3.1 Drei Schichten

- **Anwendung und API:** Der Spielcode definiert Komponenten, Systeme und Szenen. Die Programmierschnittstelle (API) stellt dafür die nutzbaren Funktionen und Datentypen bereit. JavaScript bindet die Anwendung in den Browser ein und verarbeitet Eingaben.
- **Engine-Kern:** Der Rust-Kern verwaltet Entities und Komponenten, führt Systeme aus und wird als WebAssembly im Browser ausgeführt.
- **Renderer:** JavaScript übergibt die vorbereiteten Renderdaten an WebGPU. Die GPU zeichnet daraus das Bild.

Die Schichten bleiben über einfache IDs und Speicherinformationen verbunden. Rust kann beispielsweise die Kennnummer eines Meshes speichern, also eines geometrischen Modells, während JavaScript das zugehörige Mesh verwaltet. So muss der Engine-Kern weder Browser-APIs noch konkrete GPU-Ressourcen kennen.

3.2 Bausteine der Game Engine

Artisan stellt die wiederkehrenden Grundlagen einer Spielanwendung bereit. Dazu gehören die **World** als zentraler Datenbestand des ECS, Systeme, Zeitsteuerung, Eingaben, Transformationen für Position und Drehung, Kameras sowie Ressourcen für globale Einstellungen.

Spielcode bindet diese Bausteine ein und definiert eigene Komponenten und Systeme. Für die Simulation **Vivarium** sind das beispielsweise Siedler, Ressourcen und Geländeflächen. Die Bausteine der Engine bleiben dabei unabhängig vom konkreten Spiel.

3.3 Wo die Komponenten im Speicher liegen

Entities und Komponenten werden in Tabellen verwaltet, den **Archetypes**. Jede vorkommende Kombination von Komponenten bildet eine eigene Tabelle.

Innerhalb einer Tabelle liegt jeder Komponententyp in einer zusammenhängenden Spalte, einem `BlobVec`. Ein `BlobVec` ist ein Array für Daten beliebigen Typs. Alle Werte einer Spalte liegen lückenlos hintereinander im Speicher. Ein Entity entspricht einer Zeile über alle Spalten seiner Tabelle.

Komponenten erhalten beim Registrieren eine eindeutige Typ-ID. Ein Entity besteht aus einer numerischen ID und einer Generation. Die Generation unterscheidet ein neu erzeugtes Entity von einem früheren Entity, das dieselbe ID verwendet hat. Eine globale Tabelle ordnet jedem Entity seine aktuelle Tabelle und seine Zeilennummer zu.

Wird ein Entity gelöscht, rückt das letzte Entity der Tabelle in die frei gewordene Zeile nach (**Swap-Remove**). Dadurch entstehen keine Lücken im Speicher und die dichte Anordnung bleibt ohne aufwendiges Aufräumen erhalten.

3.4 Wie Daten abgefragt werden

Systeme greifen über **Queries** auf Komponenten zu. Eine Query beschreibt, welche Komponenten gelesen oder verändert werden sollen, zum Beispiel `Query<(&mut Position, &Velocity)>`.

Beim ersten Ausführen ermittelt die Query alle Tabellen, die die geforderten Komponenten enthalten. Sie speichert diese Tabellen in einem Cache. Bei folgenden Durchläufen liest das System die passenden Tabellen direkt aus dem Cache und läuft spaltenweise durch die Daten.

Queries unterstützen Filter: `With<T>` verlangt das Vorhandensein einer Komponente, `Without<T>` schließt Entities mit dieser Komponente aus. Wird eine Schleife parallel ausgeführt, teilt Rayon die Zeilen der Spalten auf mehrere Workerthreads auf.

3.5 Wer wann rechnen darf

Systeme können als lesend oder schreibend auf Komponenten deklariert werden. Der **Scheduler** ordnet die Ausführung:

1. Er prüft, welche Systeme dieselben Komponenten anfordern.
2. Systeme, die dieselben Daten nur lesen, dürfen gleichzeitig laufen.
3. Ein System, das eine Komponente verändert, läuft getrennt von allen anderen Systemen, die auf dieselbe Komponente zugreifen.

Der Scheduler teilt Systeme anhand dieser Zugriffsregeln in Ausführungsstufen ein, sogenannte **Stages**. Alle Systeme innerhalb einer Stage laufen parallel über den Thread-Pool von Rayon. Erst wenn alle Systeme einer Stage beendet sind, beginnt die nächste Stage.

3.6 Wie Daten zu JavaScript kommen

Um Simulationsdaten darzustellen, muss JavaScript die berechneten Werte an WebGPU übergeben. Ein Kopieren jedes einzelnen Werts über die WebAssembly-Grenze hinweg wäre bei hunderttausenden Entities zu langsam.

Artisan nutzt stattdessen eine direkte **Speicherbrücke**: Rust schreibt die vorbereiteten Renderdaten, also Positionen, Farben und Instanzdaten, in einen zusammenhängenden Bereich des WebAssembly-Speichers. JavaScript erzeugt ein `TypedArray`, das direkt auf diesen Speicherbereich verweist, ohne die Daten zu kopieren (**Zero-Copy**). Erst für die Übergabe an WebGPU werden die Daten in einem Schritt in den Speicher der Grafikkarte übertragen (`writeBuffer`).

3.7 Wie WebGPU zeichnet

Der Renderer sammelt Renderbefehle und fasst gleichartige Objekte zusammen (**Batching**):

1. Ein Mesh-System sammelt alle Entities mit sichtbaren Komponenten und sortiert sie nach Mesh und Material.
2. Für jede Gruppe werden die Instanzdaten im WebAssembly-Speicher aufbereitet.
3. JavaScript übergibt die Daten gebündelt an WebGPU.
4. WebGPU zeichnet jede Gruppe mit einem einzigen instanziierten Draw Call.

Pipeline-Zustände wie Shader und Tiefentests werden beim Start der Anwendung erzeugt und zur Laufzeit wiederverwendet. Ein einfacher Sichtbarkeitstest (**Frustum Culling**) schließt Objekte außerhalb des Kamerasichtfelds vor der Übergabe an die Grafikkarte aus.

4 Anwendungsfall: Vivarium

Vivarium ist die zusammenhängende Anwendungs-Demo von Artisan. Die Simulation erzeugt einen Planeten, verteilt Ressourcen auf seiner Oberfläche und simuliert autonome Siedler. Dieselben Flächen bilden die sichtbare Welt und die räumliche Grundlage der Simulation.

4.1 Vom Mesh zur Welt

Der Planet basiert auf einer **Icosphere**, einer aus Dreiecken aufgebauten Annäherung an eine Kugel. Ausgangspunkt ist ein Ikosaeder mit 20 Dreiecksflächen. Beim **Subdivision Level 6** werden diese Flächen sechsmal unterteilt und die neuen Eckpunkte auf eine Kugel projiziert. Benachbarte Dreiecke teilen zunächst dieselben Eckpunkte, in der Computergrafik Vertices genannt. So kann Artisan ihre Nachbarschaft direkt aus dem Mesh ableiten.

Für das Rendering erhält jede Fläche anschließend eigene Vertices und kann unabhängig eingefärbt werden. Der zuvor berechnete Graph bleibt für die Simulation erhalten. Ein Graph beschreibt hier Flächen als Knoten und ihre Nachbarschaften als verbindende Kanten. Jeder Flächenindex verweist dabei auf dieselbe Position in den Daten für Höhe, Klima, Ressourcen und Bevölkerung.

4.2 Höhe, Klima und Ressourcen

Ein **Seed** legt den reproduzierbaren Ausgangspunkt der Generierung fest: Derselbe Seed erzeugt dieselbe Welt. Mehrlagiges Rauschen (**Simplex-Noise**) modelliert das Höhenrelief mit Tiefsee, Küsten, Ebenen und Gebirgen.

Die Temperatur sinkt zu den Polen und mit zunehmender Höhe. Niederschlag und Windsysteme bestimmen die Feuchtigkeit. Aus der Kombination von Temperatur und Feuchtigkeit entstehen zehn unterschiedliche **Biome**, darunter Wüsten, Grasland, Wälder und Tundra.

Jedes Biom erneuert periodisch eigene Ressourcen: Wälder liefern Holz, Grasland bietet Weideflächen, Flüsse und Küsten sichern Wasser und Nahrung.

4.3 Der Simulationsgraph

Der Simulationsgraph verbindet die Zentren benachbarter Dreiecke. Weil jedes Dreieck genau drei Nachbarn besitzt, ist der Graph regelmäßig aufgebaut.

Siedler nutzen diesen Graphen, um Wege zu finden. Flüsse und Dürren breiten sich entlang der Kanten aus. Der Graph arbeitet unabhängig von physikalischen Kollisionsberechnungen: Siedler bewegen sich von Fläche zu Fläche, statt kontinuierliche Positionsprüfungen im Raum auszuführen. Das hält den Rechenaufwand pro Siedler gering.

4.4 Autonome Siedler

Siedler sind einzelne Entities mit eigenen Komponenten: Position auf dem Graphen, Hunger, Alter, gesammelte Ressourcen und Zugehörigkeit zu einem Stamm. Ein Zustandsautomat steuert ihr Verhalten:

1. **Nahrung suchen:** Sinkt der Energievorrat, sucht der Siedler auf benachbarten Flächen nach Nahrung.
2. **Ressourcen sammeln:** Ist der Siedler gesättigt, sammelt er Holz oder Steine auf seiner aktuellen Fläche.
3. **Zurückkehren:** Mit vollem Inventar kehrt der Siedler zur Siedlung seines Stammes zurück und lagert die Vorräte ein.
4. **Bauen und Vermehren:** Bei ausreichend Vorräten wächst die Siedlung. Neue Gebäude entstehen, neue Siedler werden erzeugt.
5. **Sterben:** Siedler altern und sterben; verhungerte Siedler scheiden ebenfalls aus der Simulation aus.

4.5 Was Vivarium zeigt

Vivarium führt die Teile von Artisan zusammen:

- Die Planetenflächen liegen als zusammenhängende Daten im Speicher.
- Siedler werden als ECS-Entities mit Komponenten für Position, Inventar und Zustand simuliert.
- Systeme für Hunger, Bewegung, Ressourcenregeneration und Alterung laufen getaktet ab.
- Das WebGPU-Rendering liest dieselben Flächendaten über die Zero-Copy-Brücke und stellt den Planeten mit allen Siedlern im Browser dar.

5 Evaluation und Performanzmessungen

Die Evaluation vergleicht Artisan, Bevy ECS und Flecs anhand vorab festgelegter Arbeitslasten aus Spielen und Simulationen.

Ein Benchmark ist ein kontrolliertes Messprogramm. Seine Arbeitslast legt genau fest, welche Operationen in welchem Umfang ausgeführt werden. Die Gesamtheit aller hier verwendeten Arbeitslasten wird im Folgenden als Testsuite bezeichnet.

5.1 Methodik

5.1.1 Auswahl der Tests

Die Kategorien wurden aus verbreiteten ECS-Benchmarks und typischen Abläufen in Spielen und Simulationen abgeleitet. Sie umfassen die dichte Iteration durch zusammenhängende Komponentenspalten, unterschiedlich verteilte Archetypes, das Erzeugen und Löschen von Entities, Änderungen ihrer Komponentenkombination, den Zugriff auf einzelne Entities, Änderungserkennung und die Ausführung mehrerer Systeme durch den Scheduler. Damit enthält die Suite sowohl günstige als auch ungünstige Fälle für archetypbasierte Speicherung.

Kategorien und Arbeitsumfänge standen vor der ersten ausgewerteten Messung fest. Danach wurde kein Test ergänzt, entfernt oder inhaltlich verändert. Sämtliche gültigen Ergebnisse werden berichtet.

5.1.2 Durchführung

Alle drei Engines werden seriell, also auf jeweils einem Ausführungsthread, gemessen. Dadurch beeinflussen unterschiedliche Thread-Pools das Ergebnis nicht. Drei verworfene Aufwärmäufe bringen Code und Speicherpfade zunächst in einen wiederholbaren Zustand. Danach folgen 15 Messläufe mit jeweils neu aufgebauter ECS-World, dem zentralen Datenbestand der jeweiligen Engine. Jede Kategorie berechnet außerdem eine Prüfsumme aus ihrem Ergebnis. Stimmen diese Kontrollwerte nicht überein, haben die Engines nicht dieselbe Arbeit ausgeführt und die Messung wird ausgeschlossen.

5.1.3 Auswertung

Angegeben wird der Median, also der mittlere Wert der 15 nach Laufzeit sortierten Messläufe. Ein verteilungsfreies 95-Prozent-Konfidenzintervall aus den geordneten Einzelmessungen beschreibt zusätzlich die Unsicherheit dieses Medians, ohne eine Normalverteilung vorauszusetzen. Die Einzelmessungen bleiben in den Ergebnisdateien erhalten.

5.2 ECS-Messreihe

Die Tests untersuchen nicht nur abstrakte ECS-Operationen, sondern typische Bestandteile eines Simulationsschritts. Eine Iteration über Position und Geschwindigkeit entspricht etwa der Bewegung vieler Figuren pro Frame, also pro berechnetem Bild. Das Erzeugen und Löschen von Entities bildet sogenannte Spawn- und Despawn-Ereignisse ab; Komponentenwechsel entsprechen beispielsweise dem Hinzufügen eines Status oder dem Wechsel eines Zustands.

5.2.1 Gesamtergebnis

Artisan

1,05×

schnellste in 20 von 34

Flecs

1,37×

schnellste in 12 von 34

Bevy 0.18.1

1,50×

schnellste in 2 von 34

Der Wert fasst zusammen, wie weit eine Engine im Mittel vom jeweils schnellsten Ergebnis einer Kategorie entfernt liegt. 1,00 bedeutet, dass sie in allen Kategorien am schnellsten war; höhere Werte bedeuten einen größeren durchschnittlichen Abstand. Für diese Verhältnisse wird das geometrische Mittel verwendet: Es mittelt die Faktoren durch Multiplikation und anschließendes Wurzelziehen, damit nicht eine einzelne Kategorie mit besonders großem Zahlenwert die Zusammenfassung bestimmt.

5.3 Messdiagramme

5.3.1 Iteration nach Entity-Zahl (Sweep A)

Gemessen wird, wie die Laufzeit beim Verarbeiten wachsender Entity-Mengen (1.000 bis 1.000.000) steigt. Beide Achsen sind logarithmisch, da die Entity-Zahlen drei Größenordnungen umfassen.

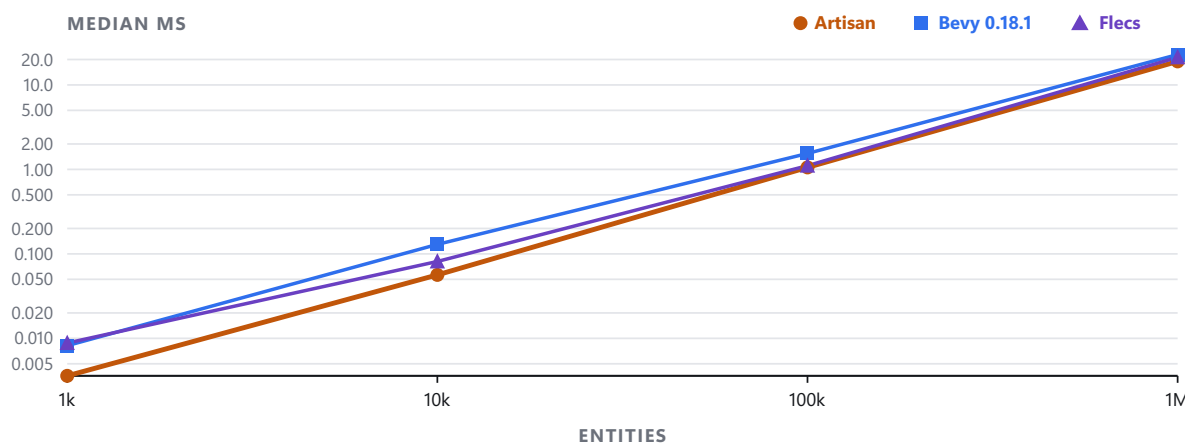


Abbildung 3: A2: Eine Komponente lesen, eine schreiben (z. B. Geschwindigkeit lesen, Position schreiben).

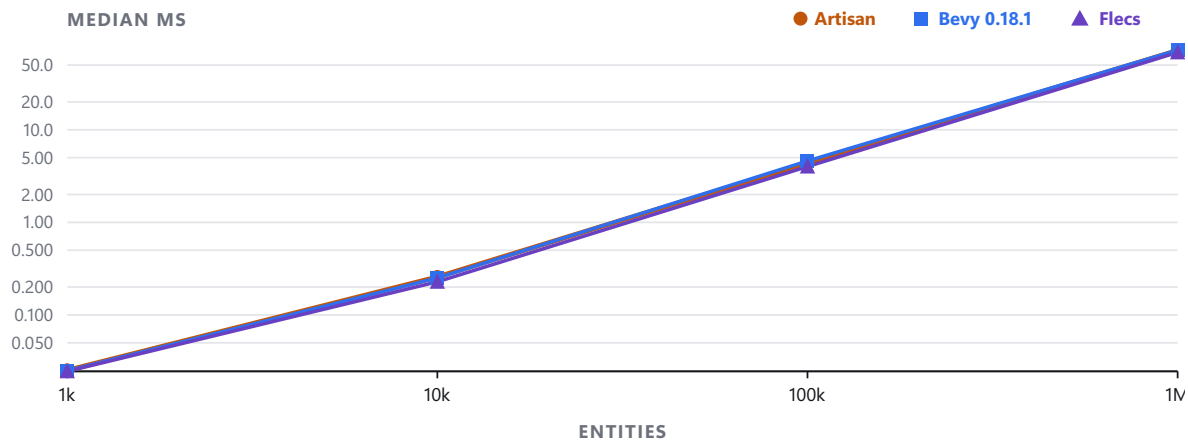


Abbildung 4: A4: Acht Komponenten (komplexes System verarbeitet mehrere Zustände).

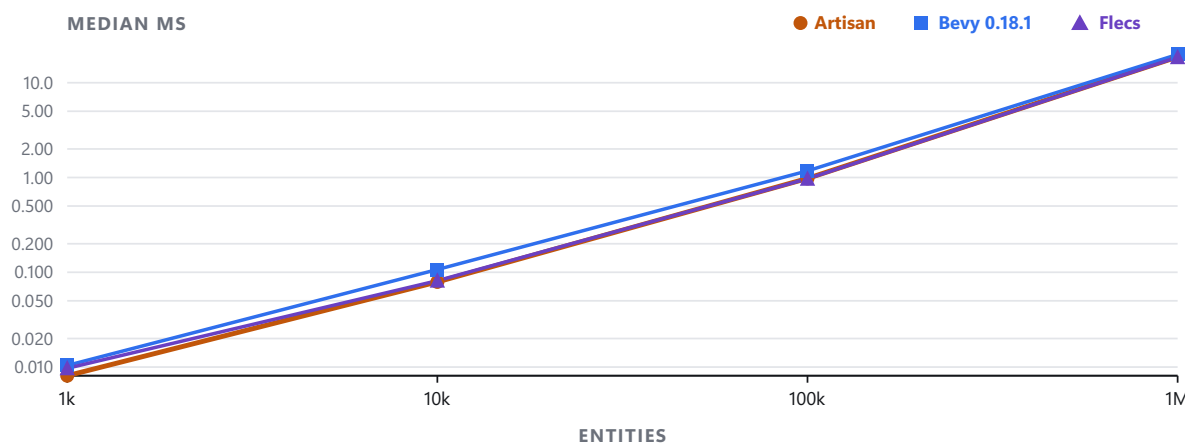


Abbildung 5: A5: Nur lesen, zwei Komponenten (Auswertung ohne Schreibvorgang).

5.3.2 Archetype-Fragmentierung (Sweep B)

Dieselben 100.000 Entities werden auf immer mehr Archetypes verteilt. Diese Archetype-Fragmentierung entspricht einer Spielwelt mit zunehmend unterschiedlichen Kombinationen von Komponenten und damit mehr getrennten Speicherbereichen.

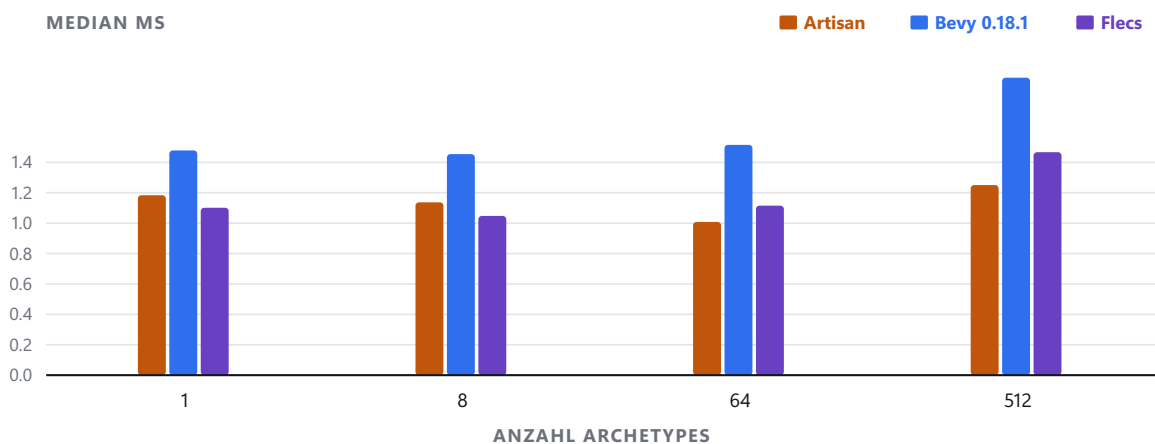


Abbildung 6: B1: Zahl der Archetypes (1 bis 32), auf die dieselben 100.000 Entities verteilt sind.

5.4 Alle 34 Messkategorien im Detail

Median aus 15 Wiederholungen, Angaben in Millisekunden. Weniger ist besser.

Kategorie	Artisan	Bevy 0.18.1	Flecs	Bester
Iteration über Komponenten				
1 Komponente schreiben · 1.000	0,003 ms	0,008 ms	0,009 ms	Artisan
1 lesen, 1 schreiben · 1.000	0,004 ms	0,008 ms	0,009 ms	Artisan
4 Komponenten · 1.000	0,013 ms	0,013 ms	0,015 ms	Bevy
8 Komponenten · 1.000	0,025 ms	0,025 ms	0,025 ms	Flecs
2 Komponenten, nur lesen · 1.000	0,008 ms	0,010 ms	0,010 ms	Artisan
1 Komponente schreiben · 10.000	0,038 ms	0,111 ms	0,076 ms	Artisan
1 lesen, 1 schreiben · 10.000	0,056 ms	0,130 ms	0,081 ms	Artisan
4 Komponenten · 10.000	0,133 ms	0,128 ms	0,127 ms	Flecs
8 Komponenten · 10.000	0,257 ms	0,251 ms	0,228 ms	Flecs
2 Komponenten, nur lesen · 10.000	0,079 ms	0,107 ms	0,082 ms	Artisan
1 Komponente schreiben · 100.000	0,597 ms	1,159 ms	0,767 ms	Artisan
1 lesen, 1 schreiben · 100.000	1,056 ms	1,544 ms	1,107 ms	Artisan
4 Komponenten · 100.000	2,168 ms	1,959 ms	2,283 ms	Bevy
8 Komponenten · 100.000	4,375 ms	4,605 ms	4,005 ms	Flecs
2 Komponenten, nur lesen · 100.000	0,978 ms	1,175 ms	0,969 ms	Flecs
1 Komponente schreiben · 1 Mio.	8,051 ms	14,491 ms	9,446 ms	Artisan
1 lesen, 1 schreiben · 1 Mio.	18,993 ms	22,871 ms	21,215 ms	Artisan
4 Komponenten · 1 Mio.	36,187 ms	38,242 ms	36,160 ms	Flecs
8 Komponenten · 1 Mio.	71,918 ms	72,668 ms	68,548 ms	Flecs
2 Komponenten, nur lesen · 1 Mio.	18,705 ms	19,870 ms	18,587 ms	Flecs
Verteilung auf Archetypes				
1 Archetype · 100.000 Entities	1,184 ms	1,478 ms	1,101 ms	Flecs
8 Archetypes · 100.000 Entities	1,137 ms	1,454 ms	1,048 ms	Flecs
64 Archetypes · 100.000 Entities	1,008 ms	1,515 ms	1,115 ms	Artisan
512 Archetypes · 100.000 Entities	1,251 ms	1,957 ms	1,467 ms	Artisan
Entities erzeugen und löschen				
Leere Entity erzeugen · 200.000	1,304 ms	5,197 ms	2,062 ms	Artisan
Entity mit 2 Komponenten erzeugen · 200.000	5,142 ms	9,980 ms	18,133 ms	Artisan
Entity löschen · 200.000	2,449 ms	5,960 ms	2,665 ms	Artisan
Komponenten ändern				
Komponente hinzufügen · 100.000	1,905 ms	3,816 ms	2,829 ms	Artisan
Komponente entfernen · 100.000	1,851 ms	4,232 ms	2,811 ms	Artisan
Hinzufügen und Entfernen im Wechsel · 20.000 × 20	11,236 ms	25,021 ms	18,943 ms	Artisan
Wahlfreier Zugriff				
Wahlfreier Lesezugriff · 100.000 × 10	8,323 ms	9,303 ms	5,900 ms	Flecs
Wahlfreier Schreibzugriff · 100.000 × 10	10,500 ms	13,998 ms	6,015 ms	Flecs
Änderungserkennung				
Vereinzelte Änderungen · 200.000	0,874 ms	1,671 ms	—	Artisan
Scheduler				
3 Systeme · 100.000	3,024 ms	4,984 ms	3,511 ms	Artisan

5.5 Wo Artisan verliert

Auch Kategorien, in denen eine andere Implementierung schneller ist, werden vollständig ausgewiesen. Nur so lässt sich erkennen, für welche Arbeitslasten die Architektur Vorteile oder Nachteile besitzt.

Kategorie	Gruppe	Artisan ist	Schnellste
Wahlfreier Schreibzugriff · 100.000 × 10	Wahlfreier Zugriff	1,75x langsamer	Flecs
Wahlfreier Lesezugriff · 100.000 × 10	Wahlfreier Zugriff	1,41x langsamer	Flecs
8 Komponenten · 10.000	Iteration über Komponenten	1,13x langsamer	Flecs
4 Komponenten · 100.000	Iteration über Komponenten	1,11x langsamer	Bevy 0.18.1
8 Komponenten · 100.000	Iteration über Komponenten	1,09x langsamer	Flecs
8 Archetypes · 100.000 Entities	Verteilung auf Archetypes	1,09x langsamer	Flecs
1 Archetype · 100.000 Entities	Verteilung auf Archetypes	1,07x langsamer	Flecs
8 Komponenten · 1 Mio.	Iteration über Komponenten	1,05x langsamer	Flecs
4 Komponenten · 10.000	Iteration über Komponenten	1,05x langsamer	Flecs
8 Komponenten · 1.000	Iteration über Komponenten	1,02x langsamer	Flecs
2 Komponenten, nur lesen · 100.000	Iteration über Komponenten	1,01x langsamer	Flecs
4 Komponenten · 1.000	Iteration über Komponenten	1,01x langsamer	Bevy 0.18.1
2 Komponenten, nur lesen · 1 Mio.	Iteration über Komponenten	1,01x langsamer	Flecs
4 Komponenten · 1 Mio.	Iteration über Komponenten	1,00x langsamer	Flecs

5.6 Testumgebung

Prozessor	Intel Core i5-13600KF (14 Kerne, 20 Threads)
Grafikkarte	NVIDIA GeForce RTX 4070 (12 GB VRAM)
Betriebssystem	Windows 11 Pro (10.0.26200)
Arbeitsspeicher	32 GB DDR5-6000
Rust-Version	rustc 1.93.0-nightly (b6d7ff3aa 2025-11-14)
Profil	release, opt-level 3
Bevy-Version	0.18.1
Flecs-Version	4.1.5 (msvc)

6 Demos und Demonstratoren

Alle Demos sind interaktiv und lassen sich ohne Installation direkt im Browser starten. Gemeinsam zeigen sie unterschiedliche Teile der Engine - von der vollständigen Zivilisationssimulation bis zu isolierten ECS-, Scheduler- und GPU-Pfaden.

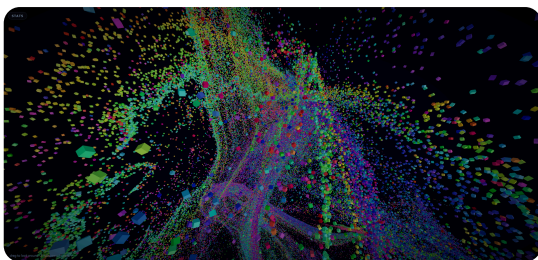
6.1 Demoübersicht



Vivarium

Die planetare Zivilisationssimulation verbindet prozedurale Weltgenerierung, individuelle Siedler, Ressourcen, Simulationssysteme und WebGPU-Rendering in einer fortlaufenden Anwendung. Startpopulation und Seed lassen sich vor dem Start wählen.

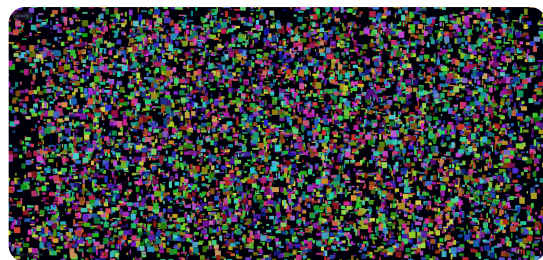
[Im Browser öffnen](#)



Murmuration

Viele ECS-Entities folgen einem Strömungsfeld. Ein GPU-Modus stellt CPU- und GPU-Simulation direkt gegenüber.

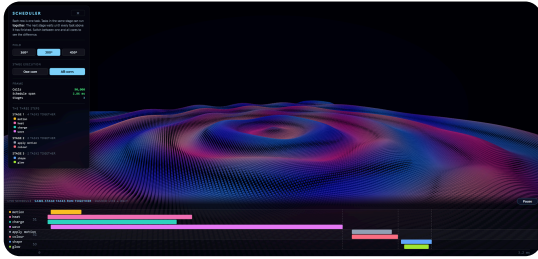
[Im Browser öffnen](#)



Bouncing Rects

Rechtecke werden in einem GPU-Storage-Buffer durch einen Compute Shader bewegt und instanziiert gezeichnet.

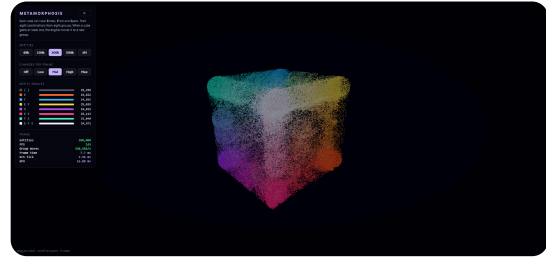
[Im Browser öffnen](#)



Scheduler

Eine Zeitansicht macht parallele Stufen und Zugriffskonflikte sichtbar; die Ausführung lässt sich auf einen Thread begrenzen.

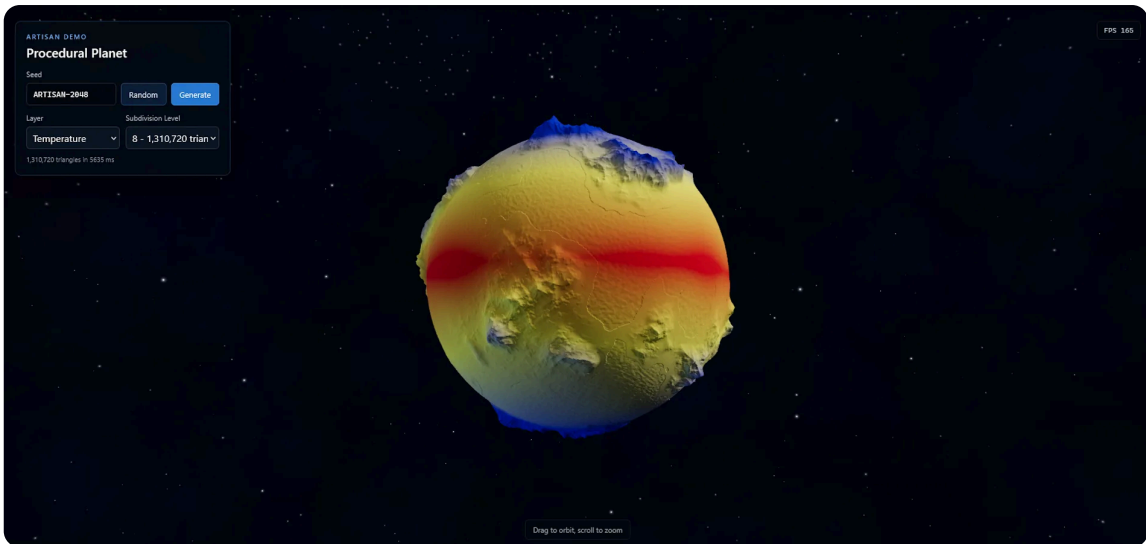
[Im Browser öffnen](#)



Metamorphosis

Drei optionale Komponenten erzeugen acht Signaturen und zeigen strukturelle Wechsel zwischen Archetype-Tabellen.

[Im Browser öffnen](#)



Procedural Planet

Ein prozedural erzeugter Planet lässt sich mit verschiedenen Seeds, Subdivision Levels und Daten-Layern untersuchen.

[Im Browser öffnen](#)

7 Grenzen

Der Prototyp belegt, dass der Ansatz funktioniert. Er belegt noch nicht, dass Artisan vollständig, auf jedem Gerät gleich leistungsfähig oder grundsätzlich schneller als etablierte ECS-Implementierungen ist.

7.1 API- und Speicherverträge

Ein Entity-Handle ist die Kennung, über die Programmteile später wieder auf ein Entity verweisen. Rust-interne Handles bestehen aus einer numerischen ID und einer Generation. Gelöschte IDs können wiederverwendet werden; die erhöhte Generation unterscheidet das neue Entity von einem alten Handle. Mehrere JavaScript-Schnittstellen übertragen jedoch nur die numerische ID. Ein auf JavaScript-Seite gespeichertes Handle ist nach Wiederverwendung seiner ID daher nicht durch denselben Generationstest geschützt. Für eine robuste öffentliche API müsste JavaScript ebenfalls ID und Generation erhalten.

Ein Speicherzeiger bezeichnet die Adresse eines Werts. Solche direkten Zeiger und die darauf angelegten TypedArrays können durch das Wachstum einer Komponentenspalte, einen Archetype-Wechsel oder das Wachstum des WebAssembly-Speichers ungültig werden. Der JavaScript-Query-Cache bewahrt bereits berechnete Speicheransichten zur Wiederverwendung auf. Pro Anfrage vergleicht er einen Strukturzähler, der Änderungen an der ECS-Anordnung erfasst, und die Identität des zugrunde liegenden Speicherbuffers. Passen diese Werte nicht mehr, liegt ein Cache Miss vor und die betroffenen Arrays werden neu aufgebaut. Einzelne außerhalb dieses Caches gespeicherte Zeiger bleiben dennoch an dieselben Lebensdauerregeln gebunden. Diese Regeln sind beherrschbar, machen die direkte Speicherbrücke aber fehleranfälliger als eine vollständig gekapselte Schnittstelle.

7.2 Laufzeit und Korrektheit

Die zeilenbezogene Änderungserkennung setzt einen nachverfolgten Schreibpfad voraus, der jede veränderte Zeile ausdrücklich markiert. Ein gewöhnlicher veränderlicher Query-Zugriff, in Rust als `&mut T` geschrieben, aktualisiert nur den Änderungswert der gesamten Spalte. Er markiert nicht jede tatsächlich geschriebene Zeile. Filter, die eine genaue Aussage pro Entity benötigen, müssen deshalb einen der nachverfolgten Schreibpfade verwenden.

Systeme können Labels tragen und über Vorher-/Nachher-Abhängigkeiten geordnet werden. Der Scheduler berechnet daraus eine topologische Reihenfolge, in der jede angegebene Abhängigkeit vor ihrem Nachfolger liegt. Diese Logik bleibt jedoch unvollständig: Bei kompatiblen Datenzugriffen können zwei so geordnete Systeme anschließend in derselben parallelen Stage landen, also in einer Gruppe gleichzeitig ausgeführter Systeme. Eine belastbare Ordnung allein durch Labels erfordert daher eine Korrektur des Schedulers.

7.3 Aussagekraft der Benchmarks

In der festgelegten seriellen Suite ist Artisan in 20 von 34 Kategorien am schnellsten. Im geometrischen Mittel liegt es näher am jeweils schnellsten Ergebnis als Bevy und Flecs. Damit ist die zulässige Aussage klar: Für diese Arbeitslasten, Versionen und Konfigurationen ist Artisan eine sehr konkurrenzfähige und insgesamt schnellste Implementierung.

Daraus folgt nicht, dass Artisan allgemein schneller als Flecs oder Bevy ist. Die Suite bildet typische, aber nur ausgewählte Situationen ab und misst keine parallelen Systeme, komplexen Beziehungen, große Anwendungsgraphen oder den vollständigen Funktionsumfang der Frameworks. Außerdem unterscheiden sich Sprache und Compiler, also die Programme, die Quellcode in ausführbaren Maschinencode übersetzen. `rustc` erzeugt die beiden Messprogramme für Rust, während der Microsoft C/C++ Compiler (MSVC) das Flecs-Programm erzeugt. Unterschiede dieser Codegenerierung sind Teil des Ergebnisses.

Die Einzelwerte zeigen zudem echte Zielkonflikte. Flecs ist beim zufälligen Zugriff und bei vereinzelten Änderungen schneller, während Artisan besonders bei mehreren linearen Iterationen und strukturellen Arbeitslasten gewinnt. Die Ergebnisse sprechen deshalb für die untersuchte Implementierung und ihre Schwerpunkte, nicht für die Überlegenheit eines Speichermodells in jeder Situation.

Absolute Zeiten gelten nur für die dokumentierte Maschine. Auch relative Unterschiede sollten nur innerhalb der geprüften Größen und Szenarien übertragen werden. Für den Renderingpfad gibt es auf dieser Seite bewusst keinen Leistungsvergleich: Die Demos zeigen seine Funktionsfähigkeit und Größenordnung, ersetzen aber keinen kontrollierten Vergleich mit einer anderen Engine.

7.4 Grenzen als Web-Engine

WebAssembly und WebGPU stellen standardisierte Bausteine für rechen- und grafikintensive Webanwendungen bereit. Web Workers und `SharedArrayBuffer` ermöglichen die für den Rayon-Pfad benötigte Form geteilter Arbeit. Damit sind datenintensive Spiele und Simulationen im Browser technisch realistisch [9], [13].

Der Browser bringt zugleich eigene Grenzen mit: Geräte und Implementierungen unterscheiden sich, geteilter Speicher benötigt zusätzliche Sicherheits-Header und große Anwendungen müssen ihre Assets zunächst übertragen und zwischenspeichern. WebGPU befindet sich zum Zeitpunkt dieser Arbeit außerdem noch im W3C-Candidate-Recommendation-Prozess. Das ist die fortgeschrittene Prüfphase vor einer endgültigen W3C Recommendation, in der Implementierungen und Tests weiter abgeglichen werden [13]. Verfügbarkeit, Treiberverhalten und Leistungswerte müssen deshalb auf den tatsächlich unterstützten Plattformen geprüft werden.

Hinzu kommt die Produktreife: Artisan besitzt weder den Werkzeugumfang noch die Plattformabdeckung etablierter Engines. Der Ansatz ist besonders interessant, wenn sofortige Ausführung per URL, Web-Oberflächen und große datenorientierte Simulationen wichtig sind. Er ist aber kein universeller Ersatz für native Engines oder deren ausgereifte Editoren, Konsolenunterstützung und Ökosysteme.

8 Fazit

Moderne Webtechnologien reichen für leistungsfähige browsernative Spiele und Simulationen. Artisan zeigt dies als funktionsfähiger Prototyp. Es handelt sich jedoch weder um eine vollständige Engine noch um den Nachweis einer allgemeinen Überlegenheit gegenüber nativen Systemen.

8.1 Antwort auf die Ausgangsfrage

Am Anfang stand die Frage, wie weit sich moderne Webtechnologien für Spiele treiben lassen. Dafür sollte keine bestehende native Engine lediglich in den Browser exportiert werden. Stattdessen entstand eine eigene Architektur aus einem datenorientierten Entity-Component-System, einem Rust-Kern in WebAssembly und einem WebGPU-Renderer.

Die Untersuchung beantwortet diese Frage grundsätzlich positiv. Der Rust-Kern verwaltet große, zusammenhängende Datenmengen, WebAssembly macht ihn im Browser nutzbar und WebGPU übernimmt die grafikintensiven Pfade. Die Anwendungen starten per URL und lassen sich zugleich mit gewöhnlichen Web-Oberflächen verbinden. Die Verbindung dieser Eigenschaften ist der eigentliche Nachweis des Projekts.

8.2 Was der Prototyp belegt

Mit Artisan wurden mehrere lauffähige Spiele und technische Demos umgesetzt. Vivarium verbindet eine prozedural erzeugte Planetenoberfläche mit individuellen Siedlern, lokalen Ressourcen, zeitlich geordneten Systemen und instanziiertem Rendering. Kleinere Anwendungen zeigen unter anderem strukturelle Änderungen, parallele Systemausführung, GPU-Compute und große instanziierte Szenen. Alle Anwendungen laufen ohne Installation direkt im Browser.

Die ECS-Messreihe stützt die Architekturentscheidung. In der dokumentierten seriellen Messreihe war Artisan in 20 von 34 gültigen Kategorien die schnellste der drei untersuchten Implementierungen. Im geometrischen Mittel lag sein Ergebnis nur um den Faktor 1,045 über dem schnellsten Ergebnis der jeweiligen Kategorie und damit in dieser Suite vor Bevy und Flecs. Das legt nahe, dass Artisan für die untersuchten spiel- und simulationsnahen Arbeitslasten zu den schnellen ECS-Implementierungen gehört. Ein allgemeines Urteil wie „Artisan ist schneller als Flecs“ wäre dennoch nicht haltbar: Funktionsumfang, Compiler, Plattform und nicht geprüfte Arbeitslasten unterscheiden sich. Zudem gewinnt Flecs einzelne Kategorien, insbesondere beim zufälligen Zugriff und bei vereinzelter Änderungen.

Die Demos belegen außerdem, dass ECS, WebAssembly und WebGPU gemeinsam in realen Anwendungspfaden funktionieren. Artisan unterstützt einen CPU/ECS-Pfad, dessen simulierte Entities über WebGPU instanziiert gezeichnet werden, sowie einen GPU-getriebenen Pfad, in dem Simulation und Instanzdaten auf der Grafikkarte verbleiben. In einem explorativen Lauf der Murmuration-Demo wurden auf einer GeForce RTX 4070 zehn Millionen Würfel ohne Frustum Culling, also ohne das Überspringen außerhalb des Kamerabereichs liegender Würfel, mit 60 Bildern pro Sekunde dargestellt. Das ist eine Demonstrationsbeobachtung, kein kontrollierter Engine-Vergleich und keine Leistungszusage für andere Szenen, Geräte oder Browser.

8.3 Bewertung der Architektur

Die Architektur ist überzeugend, weil sie den Browser nicht nur als Exportziel behandelt. Rust und das ECS bilden einen datenorientierten Simulationskern, WebGPU hält große parallele Arbeitslasten auf der Grafikkarte und JavaScript bindet die Engine in die Webanwendung ein. Diese Aufteilung verbindet Rechenleistung mit den Stärken des Webs: unmittelbare Verteilung, plattformunabhängiger Start und flexible Benutzeroberflächen.

Der Preis dieser Aufteilung sind zusätzliche Verträge an der WebAssembly-Grenze, Unterschiede zwischen Browsern und Geräten sowie ein noch kleines Werkzeug- und Engine-Ökosystem. Für produktive Spiele fehlen unter anderem ein ausgereifter Editor, eine stabile öffentliche API, umfassende Plattformtests und vollständig abgesicherte Speicher- und Schedulerpfade.

8.4 Ausblick

Der nächste Schritt ist nicht mehr der grundsätzliche Machbarkeitsnachweis, sondern die Verallgemeinerung: eine stabile API, sichere Handles und Speicheransichten, korrigierte Scheduler-Abhängigkeiten, bessere Werkzeuge sowie Messungen über mehrere Browser, Geräte und vollständige Spielszenen. Für den Renderer braucht es zusätzlich eine getrennte Instrumentierung seiner CPU- und GPU-Phasen, also Messpunkte, die den Zeitbedarf beider Seiten unabhängig erfassen.

Aus heutiger Sicht kann diese Architektur ein wichtiger Zukunftspfad für Spiele, Simulationen und interaktive Welten sein. Das gilt besonders dort, wo viele Entities, ein Start per URL und die Verbindung mit Web-Oberflächen zählen. Sie wird native Engines nicht pauschal ersetzen. Artisan zeigt aber, dass die Grenzen des Browsers solche Anwendungen nicht grundsätzlich ausschließen; entscheidend sind nun Zuverlässigkeit, Werkzeuge und die Breite der Plattformunterstützung.

Literaturverzeichnis

- [1] S. Mertens und Flecs Contributors, „Flecs Documentation“. Zugegriffen: 20. August 2026. [Online]. Verfügbar unter: <https://www.flecs.dev/flecs/>
- [2] R. Nystrom, *Game Programming Patterns*. Genever Benning, 2014. [Online]. Verfügbar unter: <https://gameprogrammingpatterns.com/component.html>
- [3] P. Briggs und L. Torczon, „An Efficient Representation for Sparse Sets“, *ACM Letters on Programming Languages and Systems*, Bd. 2, Nr. 1–4, S. 59–69, 1993.
- [4] Bevy Contributors, „Bevy ECS Storage Architecture“. Zugegriffen: 20. August 2026. [Online]. Verfügbar unter: https://docs.rs/bevy_ecs/0.18.1/bevy_ecs/storage/index.html
- [5] Rust Core Team, „Announcing Rust 1.0“. Zugegriffen: 20. August 2026. [Online]. Verfügbar unter: <https://blog.rust-lang.org/2015/05/15/Rust-1.0/>
- [6] A. Haas u. a., „Bringing the Web up to Speed with WebAssembly“, *ACM SIGPLAN Notices (PLDI 2017)*, Bd. 52, Nr. 6, S. 185–200, 2017, doi: 10.1145/3062341.3062363.
- [7] W3C, „WebAssembly Becomes a W3C Recommendation“. Zugegriffen: 20. August 2026. [Online]. Verfügbar unter: <https://www.w3.org/press-releases/2019/wasm/>
- [8] WebAssembly Community Group, „Understanding the WebAssembly JavaScript API“. Zugegriffen: 20. August 2026. [Online]. Verfügbar unter: <https://webassembly.org/getting-started/js-api/>
- [9] WebAssembly Community Group, „WebAssembly High-Level Goals“. Zugegriffen: 20. August 2026. [Online]. Verfügbar unter: <https://webassembly.org/docs/high-level-goals/>
- [10] Rayon Contributors, „Rayon: Data Parallelism in Rust“. Zugegriffen: 20. August 2026. [Online]. Verfügbar unter: <https://docs.rs/rayon/latest/rayon/>
- [11] I. Stepanyan, „wasm-bindgen-rayon: Rayon builder for WebAssembly“. Zugegriffen: 20. August 2026. [Online]. Verfügbar unter: <https://github.com/RReverser/wasm-bindgen-rayon>
- [12] Khronos Group, „WebGL Specification 1.0“. Zugegriffen: 20. August 2026. [Online]. Verfügbar unter: <https://registry.khronos.org/webgl/specs/latest/1.0/>
- [13] W3C GPU for the Web Working Group, „WebGPU Specification“. Zugegriffen: 20. August 2026. [Online]. Verfügbar unter: <https://www.w3.org/TR/webgpu/>
- [14] W3C GPU for the Web Working Group, „WebGPU Shading Language (WGSL)“. Zugegriffen: 20. August 2026. [Online]. Verfügbar unter: <https://www.w3.org/TR/WGSL/>

A Anhang: Nutzung KI-basierter Anwendungen

Bei der Entwicklung von Artisan und beim Schreiben dieser Dokumentation wurden generative KI-Werkzeuge als Assistenz eingesetzt. Sie unterstützten die Analyse und Überarbeitung von Code, die Fehlersuche, die Recherche sowie die sprachliche und strukturelle Überarbeitung von Texten.

A.1 Verwendete Werkzeuge

Anbieter oder Dienst	Verwendete Modelle	Schwerpunkt der Nutzung
OpenAI	GPT-5.6-sol	Sprachliche und strukturelle Überarbeitung, Verständlichkeitsprüfung, Programmierung, Codeanalyse und Fehlersuche
Anthropic	Claude Sonnet 5 und Fable 5	Programmierung, Codeanalyse, Fehlersuche und Optimierung
Google	Gemini 3.1 Pro und Gemini 3.5 Flash	Programmierung, Codeanalyse und Fehlersuche
Perplexity	Perplexity	Recherche und Auffinden möglicher Quellen

A.2 Art der Unterstützung

Bei der Softwareentwicklung halfen die Werkzeuge dabei, vorhandene Quelltexte und Schnittstellen schneller zu verstehen, Fehlermeldungen einzuordnen und Implementierungen zu überarbeiten. Dies betraf insbesondere das Entity-Component-System, die Verbindung zwischen Rust, WebAssembly und JavaScript, den WebGPU-Renderer, die planetare Simulation sowie die zugehörigen Demonstratoren und Messwerkzeuge. Vorschläge der KI-Systeme wurden an die bestehende Architektur angepasst und anhand des ausgeführten Programms, des Quellcodes und der Messdaten geprüft.

Beim Schreiben dienten die Werkzeuge dazu, unklare Formulierungen, Widersprüche, Wiederholungen und fehlende Erklärungen zu erkennen. Sie unterstützten außerdem die Neuordnung der Dokumentation und die Anpassung der technischen Erklärungen an Leser mit Grundkenntnissen in Web- und Spieleentwicklung. Perplexity wurde ergänzend zur Recherche eingesetzt. KI-Ausgaben wurden nicht als fachliche Quellen verwendet. Übernommene technische Aussagen wurden anhand der jeweiligen Originalquellen oder der tatsächlich vorhandenen Implementierung geprüft.

A.3 Eigenanteil und Verantwortung

Grundidee, Zielsetzung und Aufbau von Artisan sowie die zentralen Architekturentscheidungen wurden vom Verfasser entwickelt. Dazu gehören die Aufteilung zwischen Rust, WebAssembly, JavaScript und WebGPU, das Entity-Component-System, der Renderingpfad, die planetare Simulation, die Auswahl und Umsetzung der fünf Demonstratoren sowie die Planung und Durchführung der Messungen.

Die KI-Werkzeuge hatten eine unterstützende Funktion. Code- und Formulierungsvorschläge wurden geprüft, angepasst oder verworfen. Die fachlichen Entscheidungen sowie die Verantwortung für die endgültige Software, die dargestellten Messwerte und den Inhalt der Dokumentation blieben beim Verfasser.